Lab 5: Static Code Analysis

Project: Pharmacy Manager | Tools: Pylint, Bandit, Flake8

1. Issue Table

Issue	Tool / Type	Line(s)	Description	Fix Approach
Use of eval()	Bandit / Security	Use the line number shown for B307 in the Bandit before report.	Evaluating a string as Python code can execute unintended or attacker-controlled expressions.	Replace unrestricted eval() with a restricted expression parser and an allowlist of supported operators.
MD5 hashing	Bandit / Security	Use the line number shown for the MD5 finding in the Bandit before report.	MD5 is cryptographically weak and unsuitable for security-sensitive hashing.	Use an appropriate modern hash; for passwords, use a dedicated password-hashing scheme.
Unsafe pickle deserialization	Bandit / Security	Use the line number shown for the pickle finding in the Bandit before report.	Loading untrusted pickle data can execute code during deserialization.	Use JSON for inventory data and validate the loaded data structure.
Use of os.system()	Bandit / Security	Use the line number shown for the os.system finding in the Bandit before report.	Shell command execution can introduce command-injection risks.	Avoid shell execution where possible; use safer APIs or subprocess argument lists when needed.
Hard-coded password	Bandit / Security	Use the line number shown for the hard-coded credential finding in the Bandit before report.	Credentials embedded in source code can be exposed through source control.	Read credentials from environment variables and never commit real secrets.
Security-sensitive random values	Bandit / Security	Use the line number shown for the random-module finding in the Bandit before report.	The standard pseudo-random generator is not intended for security tokens.	Use Python's secrets module for security-sensitive random values.
Bare except clause	Pylint / Reliability	Use the line number shown for the bare-except finding in the Pylint before report.	Catching every exception can hide unexpected failures and make debugging difficult.	Catch specific expected exceptions and handle or report them appropriately.
Imports inside functions / global statement	Pylint / Maintainability	Use the line numbers shown for C0415 and W0603 in the Pylint before report, if present.	These patterns can reduce clarity and trigger Pylint warnings.	Move imports to module scope and update the inventory dictionary through explicit operations.
Formatting and style violations	Flake8 / Style	Use the line numbers listed in the Flake8 before report.	Whitespace, layout, and other style violations reduce consistency and readability.	Apply the reported PEP 8 fixes and rerun Flake8 to verify.